Atelier 1 : JavaScript, les bases

Le contexte

GreenCart est une petite boutique en ligne de produits locaux. Des producteurs de la région y vendent des fruits, des légumes, du fromage et du miel.

Dans cet atelier, vous écrivez le cœur de calcul de la boutique : le prix TTC, le total du panier, les frais de livraison, la recherche dans le catalogue. Tout se passe dans la console, avec Node. Pas de page web pour l'instant.

Vous avez déjà vu tout ce qu'il faut dans le cours JavaScript : variables, conditions, boucles `for`, fonctions, méthodes de String, tableaux et objets. Cet atelier sert à le pratiquer sur un cas concret.

Les règles de la boutique, valables pour tous les exercices :

- Les prix du catalogue sont **HT**. La TVA sur l'alimentaire est de **5,5 %**.
- La livraison coûte **4,90 €**. Elle est **offerte à partir de 40 € TTC**. Un panier vide (0 €) ne se livre pas : pas de frais.
- Une quantité commandée est un **nombre entier**, au moins 1, au plus le stock.
- Tous les montants sont **arrondis au centime**.

Les données sont dans le dossier `donnees/` : `produits.json` (12 produits) et `producteurs.json` (4 producteurs).

Comment travailler

Ouvrez un terminal **dans ce dossier**. Chaque exercice a son fichier. Les fonctions y sont déjà déclarées, avec un commentaire qui dit ce qu'elles doivent faire. Vous écrivez leur corps.

Pour essayer un exercice, lancez son fichier :

```
node exercice-1-prix.js
```

En bas de chaque fichier, le bloc « Vos essais » affiche quelques résultats. Tant que la fonction n'est pas écrite, il affiche `undefined`. Vous pouvez y ajouter vos propres `console.log`.

Pour vérifier tous les exercices :

```
node verifier.js
```

Le vérificateur affiche une ligne par exercice : « OK », ou « À revoir » suivi de ce qui ne va pas. Lancez-le souvent. Ne le modifiez pas.

Faites les exercices dans l'ordre. Les exercices 1 et 2 sont **indispensables**. Les exercices 3 et 4 sont **souhaitables**. L'exercice 5 est un **bonus**.

Exercice 1 : le prix TTC (indispensable)

Fichier : `exercice-1-prix.js`

Ce qui est demandé. Écrivez deux fonctions.

- `arrondirAuCentime(montant)` arrondit un montant au centime le plus proche.
- `prixTTC(prixHT)` calcule le prix TTC (prix HT plus 5,5 % de TVA), arrondi au centime. Elle se sert de `arrondirAuCentime`.

Rappel. Voir le cours JavaScript : les variables et `const`, slides 22 à 26, les opérateurs, slides 27 et 28, les fonctions et `return`, slides 36 à 42. L'exercice 6 de la slide 90 calculait déjà un prix TTC.

Exemple de résultat attendu.

```
arrondirAuCentime(3.376)   donne 3.38
arrondirAuCentime(2.531)   donne 2.53
prixTTC(10)                donne 10.55
prixTTC(3.2)               donne 3.38
```

Indice. 5,5 % s'écrit `0.055` en JavaScript. Pour arrondir au centime, on passe en centimes, on arrondit à l'entier, puis on revient en euros : `Math.round(montant * 100) / 100`. `Math.round(337.6)` donne `338`.

À rendre. Le fichier `exercice-1-prix.js` complété.

Exercice 2 : le panier et la livraison (indispensable)

Fichier : `exercice-2-panier.js`

Un panier est un tableau de lignes. Chaque ligne est un objet avec le nom du produit, son prix HT pour une unité et la quantité :

```
const panier = [
  { nom: 'Pommes Reinette', prix: 3.2, quantite: 2 },
  { nom: "Miel d'acacia", prix: 10.5, quantite: 3 },
];
```

Ce qui est demandé. Écrivez trois fonctions.

- `totalPanierTTC(panier)` renvoie le total TTC du panier. Calculez d'abord le total HT de toutes les lignes (prix multiplié par quantité). Appliquez ensuite la TVA **une seule fois**, sur ce total. Un panier vide coûte 0.
- `fraisLivraison(totalTTC)` renvoie 4.9, ou 0 si le total atteint 40 € TTC. Elle renvoie aussi 0 pour un total de 0 : un panier vide ne se livre pas.
- `montantAPayer(panier)` renvoie ce que le client paie : le total TTC plus la livraison, arrondi au centime.

Le haut du fichier importe déjà `prixTTC` et `arrondirAuCentime` de l'exercice 1. Servez-vous-en.

Rappel. Voir le cours JavaScript : la boucle `for`, slides 33 et 34, la condition `if`, slides 30 et 31, les comparaisons comme `>=`, slide 28, les tableaux, slides 50 à 52. L'exercice 14 de la slide 92 calculait une moyenne avec une boucle : c'est le même schéma.

Exemple de résultat attendu. Pour le panier ci-dessus :

```
totalPanierTTC(panier)   donne 39.98   (total HT 37.90)
fraisLivraison(39.98)    donne 4.9
fraisLivraison(40)       donne 0
fraisLivraison(0)        donne 0       (panier vide)
montantAPayer(panier)    donne 44.88
```

Indice. Démarrez une variable `let totalHT = 0` avant la boucle. Dans la boucle, ajoutez-lui chaque ligne : `totalHT = totalHT + ...`. Attention au cas de 40 € pile : « à partir de 40 € », c'est `>=`.

Si vous arrondissez le prix de chaque produit avant d'additionner, vous trouverez 40.00 au lieu de 39.98, et la livraison sera offerte à tort. Le vérificateur vous le signalera.

À rendre. Le fichier `exercice-2-panier.js` complété.

Exercice 3 : le catalogue (souhaitable)

Fichier : `exercice-3-catalogue.js`

Le catalogue est un tableau d'objets. Un produit ressemble à ceci :

```
{ id: 4, nom: 'Carottes', categorie: 'legumes', prix: 1.8,
  unite: 'kg', stock: 60, bio: true, producteurId: 1 }
```

Ce qui est demandé. Écrivez trois fonctions, chacune avec une boucle `for`.

- `trouverParId(liste, id)` renvoie l'élément qui a cet `id`, ou `null` s'il n'existe pas. Elle doit marcher aussi bien pour les produits que pour les producteurs.
- `rechercherParNom(catalogue, texte)` renvoie un nouveau tableau avec les produits dont le nom contient le texte. Les majuscules ne comptent pas.
- `produitsDisponibles(catalogue)` renvoie un nouveau tableau avec les produits dont le stock est supérieur à 0. Le catalogue reçu ne doit pas être modifié.

Rappel. Voir le cours JavaScript : `toLowerCase` et `indexOf`, slides 44 et 45, `push`, slide 49, les objets, slide 74. Les exercices 16 (slide 92), 31 et 32 (slide 96) parcouraient déjà un tableau pour chercher ou trier des éléments.

Exemple de résultat attendu.

```
trouverParId(catalogue, 9)             donne l'objet Tomme de chèvre
trouverParId(producteurs, 4)           donne l'objet Rucher Martin
trouverParId(catalogue, 99)            donne null
rechercherParNom(catalogue, 'pomme')   donne [Pommes Reinette, Pommes de terre]
rechercherParNom(catalogue, 'MIEL')    donne [Miel de châtaignier, Miel d'acacia]
rechercherParNom(catalogue, 'kiwi')    donne []
produitsDisponibles(catalogue)         donne 10 produits (pas les Poires Williams ni le
Yaourt)
```

Indice. `indexOf` renvoie `-1` quand le texte est absent. Mettez **les deux** en minuscules avant de comparer : le nom du produit et le texte cherché. Pour `trouverParId`, le `return null` se place **après** la boucle : si on l'atteint, c'est qu'aucun élément ne correspondait.

À rendre. Le fichier `exercice-3-catalogue.js` complété.

Exercice 4 : les modules (souhaitable)

Fichiers : `exercice-4-fiche.js` et `boutique.js`

Jusqu'ici, chaque fichier exporte ses fonctions avec `module.exports`, déjà écrit pour vous. Ici, c'est vous qui écrivez l'export et l'import.

Ce qui est demandé.

1. Dans `exercice-4-fiche.js`, importez `prixTTC` de l'exercice 1 avec `require`.
2. Écrivez `ficheProduit(produit)`, qui renvoie la fiche du produit en une ligne de texte : le nom, la mention `(bio)` si le produit est bio, le prix TTC avec deux décimales et une virgule, l'unité, et la mention `(épuisé)` si le stock vaut 0.
3. Exportez `ficheProduit` avec `module.exports`.
4. Dans `boutique.js`, importez `produitsDisponibles` (exercice 3) et `ficheProduit`, puis affichez un titre et la fiche de chaque produit disponible.

Rappel. Voir le cours Node.js : slide 14, `app.js` exporte l'application et `server.js` l'importe pour la lancer. C'est le même mécanisme. Pour le texte, voir le cours JavaScript : `replace`, slide 46, et l'exercice 21 de la slide 94.

Exemple de résultat attendu.

```
ficheProduit(carottes)          donne 'Carottes (bio) : 1,90 € TTC / kg'
ficheProduit(pommesDeTerre)     donne 'Pommes de terre : 1,58 € TTC / kg'
ficheProduit(poiresWilliams)    donne 'Poires Williams : 3,80 € TTC / kg (épuisé)'
```

Et `node boutique.js` affiche :

```
GreenCart : 10 produits disponibles
Pommes Reinette (bio) : 3,38 € TTC / kg
Pruneaux d'Agen : 8,33 € TTC / pot
Carottes (bio) : 1,90 € TTC / kg
...
Miel d'acacia (bio) : 11,08 € TTC / pot
```

Indice. La ligne d'import en haut de `exercice-2-panier.js` est un modèle. Les accolades doivent se répondre : `module.exports = { ficheProduit }` d'un côté, `const { ficheProduit } = require('./exercice-4-fiche')` de l'autre. Pour écrire 1.9 sous la forme « 1,90 », `toFixed(2)` donne le texte `'1.90'`, puis `replace('.', ',')` change le point en virgule.

À rendre. Les fichiers `exercice-4-fiche.js` et `boutique.js` complétés.

Exercice 5 : bonus, map, filter et les erreurs

Fichier : `exercice-5-bonus.js`

Cet exercice utilise des notions qui ne sont pas dans le cours JavaScript. Voici le rappel.

Rappel : `filter` et `map`. Ce sont des méthodes de tableau. On leur donne une fonction, qu'elles appliquent à chaque élément. Elles renvoient un **nouveau** tableau et ne modifient pas l'ancien.

```
const nombres = [3, 8, 12, 5];

// filter garde les éléments pour lesquels la fonction renvoie true
const grands = nombres.filter(function (n) {
  return n > 6;
});
// grands vaut [8, 12]

// map remplace chaque élément par ce que renvoie la fonction
const doubles = nombres.map(function (n) {
  return n * 2;
});
// doubles vaut [6, 16, 24, 10]
```

Rappel : `throw` et `try` / `catch`. `throw` lève une erreur et arrête la fonction. `try` / `catch` attrape l'erreur pour que le programme ne plante pas. Voir aussi le cours Node.js, slides 30 et 31.

```
function diviser(a, b) {
  if (b === 0) {
    throw new Error('division par zéro');
  }
  return a / b;
}

try {
  console.log(diviser(10, 0));
} catch (erreur) {
  console.log('Problème : ' + erreur.message); // Problème : division par zéro
}
```

Ce qui est demandé.

Partie A, sans boucle `for` :

- `produitsBio(catalogue)` renvoie les produits bio, avec `filter`.
- `nomsDesProduits(produits)` renvoie un tableau des seuls noms, avec `map`.

Partie B :

- `ajouterAuPanier(panier, produit, quantite)` ajoute la ligne `{ nom, prix, quantite }` au panier et renvoie le panier. Avant d'ajouter, elle vérifie la quantité. Elle lève une erreur avec un message clair, sans rien ajouter, si la quantité n'est pas un entier, si elle est inférieure à 1, si le produit est épuisé ou si la quantité dépasse le stock. Une quantité égale au stock est acceptée.
- `essayerAjout(panier, produit, quantite)` appelle `ajouterAuPanier` dans un `try`. Si tout va bien, elle renvoie par exemple `'2 x Radis ajouté au panier'`. Sinon, elle renvoie `'Impossible : '` suivi du message de l'erreur. Elle ne plante jamais.

Exemple de résultat attendu. Les Radis ont un stock de 3.

```
nomsDesProduits(produitsBio(catalogue))
  donne ['Pommes Reinette', 'Carottes', 'Poireaux', 'Radis',
         'Fromage de chèvre frais', 'Yaourt au lait de chèvre', "Miel d'acacia"]

essayerAjout(panier, radis, 2)     donne '2 x Radis ajouté au panier'
essayerAjout(panier, radis, 4)     donne 'Impossible : stock insuffisant, il en reste 3'
essayerAjout(panier, radis, 0)     donne 'Impossible : la quantité doit être au moins 1'
essayerAjout(panier, radis, 2.5)   donne 'Impossible : la quantité doit être un nombre
entier'
```

Vos messages d'erreur peuvent être formulés autrement. Seul le début `'Impossible : '` est imposé.

Indice. `Number.isInteger(2.5)` renvoie `false` et `Number.isInteger(3)` renvoie `true`. Faites toutes les vérifications **avant** le `push`. Dans le `catch`, le texte de l'erreur est dans `erreur.message`.

À rendre. Le fichier `exercice-5-bonus.js` complété.